



Asincronismo

Parte 1

Contenido



- ❖ ¿Qué es una petición HTTP?
- ❖ AJAX.
- ❖ Tecnologías y métodos.
- ❖ El objeto XMLHttpRequest.



¿Qué es una petición HTTP?

Una petición **HTTP** es la acción por parte del navegador de solicitar a un *servidor web* un documento o archivo, ya sea una hoja de estilos **.css**, documentos **.html**, una imagen, un archivo **.js**, etc.

Gracias a dicha petición, el navegador puede descargar estos archivos, almacenarlos en un **caché** temporal y, finalmente, mostrarlos en la página actual que lo haya solicitado.

AJAX (**A**synchronous **J**avaScript **A**nd **X**ML)



- ❖ No es un lenguaje de programación sino un conjunto de tecnologías que permiten hacer páginas de internet más interactivas.
- ❖ AJAX permite que las páginas sean capaces de actualizar partes en segundo plano, sin volver a cargar toda la página.
- ❖ La petición HTTP se realiza desde JavaScript, de forma transparente al usuario.
- ❖ AJAX se hizo popular en 2005 por Google, con **Google Suggest**.





Tecnologías utilizadas

- ❖ XHTML y CSS, para crear una presentación basada en estándares.
- ❖ DOM, para la interacción y manipulación dinámica de la presentación.
- ❖ XML, XSLT y JSON, para el intercambio y la manipulación de información.
- ❖ XMLHttpRequest, para el intercambio asíncrono de información.
- ❖ JavaScript, para unir todas las demás tecnologías.

Nota: Actualmente, se utiliza más el formato JSON que el XML, pero por razones fonéticas se mantiene el término AJAX en lugar de AJAJ.



Métodos de petición AJAX

Existen varias formas de realizar peticiones HTTP mediante AJAX, las principales son:

- ❖ **XMLHttpRequest:** el más antiguo. Nativo
- ❖ **fetch:** Nuevo sistema nativo de peticiones basado en promesas. ECMAScript 6.



XMLHttpRequest (1/5)

La piedra angular de AJAX es el objeto **XMLHttpRequest** y es utilizado para intercambiar datos con el servidor en segundo plano.

Para enviar una petición al servidor, se utilizan los métodos **open()** y **send()**.

```
xhttp.open("Método", "Url", Async);
```

Método: Especifica el tipo de pedido (GET/POST).

Url: Indica la ubicación del archivo en el servidor.

Async: true (Asincrónico); false (Sincrónico).



XMLHttpRequest (2/5)

```
xhttp.send();           //Si es GET  
xhttp.send("string");   //Si es POST
```

```
let xhttp = new XMLHttpRequest();  
xhttp.open("GET", "ajax_test.php", true);  
xhttp.send();
```

```
console.log(xhttp.responseText);
```



XMLHttpRequest (3/5)

Propiedad	Descripción
STRING <code>.responseType</code>	Define el tipo de respuesta de <code>.response</code> : <code>json</code> , <code>Blob</code> , etc. Por defecto, <code>text</code> .
OBJECT <code>.response</code>	Contenido parseado automáticamente basado en <code>.responseType</code> .
STRING <code>.responseText</code>	Respuesta de la petición como texto plano, o NULL si no se ha recibido.
STRING <code>.responseURL</code>	URL de la petición HTTP a realizar.
NUMBER <code>.readyState</code>	Número que indica en que estado se encuentra la petición (ver más adelante).
NUMBER <code>.timeout</code>	Milisegundos permitidos para realizar la petición HTTP. Por defecto, 0 (sin límite).
NUMBER <code>.status</code>	Código de error HTTP de respuesta de la petición.
STRING <code>.statusText</code>	Texto con el código de error de respuesta, legible para humanos.
BOOLEAN <code>.withCredentials</code>	Indica si la petición HTTP se está realizando con credenciales.



XMLHttpRequest (4/5)

Método	Descripción
Cabeceras HTTP	
<code>.setRequestHeader(STRING name, STRING value)</code>	Permite añadir la cabecera <code>name</code> con el valor <code>value</code> a la petición HTTP.
<code>STRING .getAllResponseHeaders()</code>	Obtiene todas las cabeceras HTTP de la respuesta de la petición.
<code>STRING .getResponseHeader(STRING name)</code>	Obtiene una cabecera HTTP concreta de la respuesta de la petición.
<code>.overrideMimeType(STRING mimetype)</code>	Permite modificar el MIME (tipo de fichero) de la petición.
Acciones	
<code>.open(STRING method, STRING url)</code>	Permite preparar una petición HTTP.
<code>.send()</code>	Envía la petición previamente preparada con <code>.open()</code> .
<code>.send(OBJECT body)</code>	Idem a la anterior, enviando datos en la petición.
<code>.abort()</code>	Cancela la petición enviada.



XMLHttpRequest (5/5)

Evento	Descripción
<code>abort</code>	Se dispara cuando una petición es cancelada.
<code>load</code>	Se dispara cuando una petición se ha completado correctamente.
<code>loadstart</code>	Se dispara cuando una petición comienza a cargar datos.
<code>loadend</code>	Se dispara cuando una petición termina (con error o sin ellos).
<code>error</code>	Se dispara cuando una petición sufre un error.
<code>timeout</code>	Se dispara cuando una petición agota el tiempo máximo.
<code>progress</code>	Se dispara (varias veces) cuando una petición recibe datos.
<code>readystatechange</code>	Se dispara cuando el valor <code>.readyState</code> cambia.



onreadystatechange (1/2)

- ❖ El evento **readystatechange** se dispara cada vez que cambia el valor de **readyState**.
- ❖ La propiedad `readyState` mantiene el estado de `XMLHttpRequest`.
- ❖ La propiedad `readyState` indica el estado de la petición.

Valor	Constante	Descripción
0	<code>XMLHttpRequest.UNSENT</code>	Estado inicial. No se ha ejecutado aún <code>.open()</code> .
1	<code>XMLHttpRequest.OPENED</code>	Se ha ejecutado <code>open()</code> , pero no se ha ejecutado aún <code>.send()</code> .
2	<code>XMLHttpRequest.HEADERS_RECEIVED</code>	Se ha ejecutado <code>send()</code> . Cabeceras recibidas.
3	<code>XMLHttpRequest.LOADING</code>	Descarga de información en proceso.
4	<code>XMLHttpRequest.DONE</code>	Descarga finalizada.



onreadystatechange (2/2)

```
let xhttp = new XMLHttpRequest();  
xhttp.open("GET", "ajax_test.php", true);  
xhttp.send();  
  
xhttp.onreadystatechange = () => {  
    if(xhttp.readyState == 4 && xhttp.status == 200){  
        console.log(xhttp.responseText);  
    }  
}
```



Ejemplos



Ejercitación